Web Pallet Manual

Bridge-driven graphics, dashboards, terminals, and ECharts

Web Pallet is a browser-backed drawing and dashboard surface driven from Python or C++ through a local bridge. It is useful for quick visualization, live telemetry, demos, terminal panes, and lightweight operator dashboards.

This manual covers the bridge, the Python client, canvas drawing primitives, UI cards and controls, terminal regions, the Pallet graph library, Apache ECharts support, live updates, replay behavior, and extension patterns.

Output file: output/pdf/web_pallet_manual.pdf

How To Use This Manual

ORIENTATION

This manual is a practical field guide for building with Web Pallet. It explains how the browser, bridge, Python clients, drawing API, UI layer, graph library, terminal regions, and ECharts helper layer fit together.

- 1-4 Orientation and setup
- 5-13 Architecture, bridge behavior, and drawing basics
- 14-22 UI overlays, cards, controls, events, and tables
- 23-35 Canvas graph library: axes, series, charts, gauges, and live updates
- 36-45 Apache ECharts: raw options, helpers, builders, and live dashboards
- 46-53 Operations: CLI, reconnects, protocol, troubleshooting, performance, and security
- 54-60 Extension patterns, API references, glossary, and release checklist

The examples assume you run commands from the repository root and open pallet.html in a browser connected to ws://localhost:8080.

Core Concepts

ORIENTATION

Web Pallet has three cooperating pieces: a browser page, a bridge process, and one or more clients. The browser renders commands. The bridge relays commands and keeps compact reconnect state. Clients send drawing, UI, terminal, and chart commands.

The usual Python entry point is Pallet.for_bridge() from python/pallet.py. Canvas charts use python/pallet_graph_lib.py. Apache ECharts dashboards use python/echarts_graph_lib.py.

- pallet.html is the visual surface and loads the browser renderer.
- python/bridge.py accepts TCP clients and browser WebSocket clients.
- python/pallet.py is the general command client.
- python/pallet_graph_lib.py adds immediate-mode charts and gauges.
- python/echarts_graph_lib.py adds browser-native ECharts charts.

Quick Start

ORIENTATION

Start the bridge, open the browser surface, connect the page, then run a Python client.

```
python python/bridge.py
# Open pallet.html in a browser.
# Connect the browser to ws://localhost:8080.
python python/pallet_graph_examples.py --demo 1
```

By default the bridge uses TCP port 9000 for drawing clients and WebSocket port 8080 for browser clients. Python examples also accept --bridge-host, --bridge-port, --width, --height, and sometimes --page.

Use PALLET_BRIDGE_HOST and PALLET_BRIDGE_PORT to set Python defaults without changing scripts.

Browser Surface

RUNTIME AND DRAWING

pallet.html loads the renderer, creates the canvas and HTML overlay layers, loads ECharts from the configured CDN, and maintains pages, cards, controls, tables, terminal regions, and chart hosts.

- The browser reports its drawing size during handshake.
- Canvas commands draw immediately on the active page.
- HTML overlay commands create DOM widgets over the drawing surface.
- Chart commands create ECharts hosts that resize independently.

Bridge Process

RUNTIME AND DRAWING

The bridge is the traffic controller. Python or C++ clients connect over TCP and browsers connect over WebSocket. The bridge forwards commands, tracks browser status, and can replay compact state to reconnecting browser pages.

- Run python/bridge.py for the Python bridge.
- The cpp/bridge.cpp source provides a native bridge implementation.
- Use --no-replay-on-connect when replay is not wanted.
- Use status commands to inspect web client counts and screen metadata.

Python Client Lifecycle

RUNTIME AND DRAWING

A Pallet object owns a TCP connection to the bridge. Use it as a context manager for short scripts or call connect() and close() explicitly for long-running processes.

```
from pallet import Pallet

with Pallet.for_bridge("127.0.0.1") as pallet:
pallet.clear("#F8FAFC")
pallet.text(30, 40, "Hello, Web Pallet")
```

Pages And Groups

RUNTIME AND DRAWING

Pages let you keep multiple dashboards alive and switch which one the browser shows. Groups let you replace a logical set of commands together, which is especially useful for panels that are rebuilt from scratch.

- set_page(page) sets default page metadata on following commands.
- show_page(page) changes the visible browser page.
- replace_group(group, commands) swaps a group atomically.
- coalesce_group(group) captures commands and replaces the group.

Batches And Metadata

RUNTIME AND DRAWING

The Python client can batch commands and attach metadata such as page or group. Batching reduces round trips and keeps related commands ordered.

```
pallet.begin_batch()
pallet.fill_rect(20, 20, 200, 120, "#DBEAFE")
pallet.text(36, 58, "Batched")
pallet.end_batch()

with pallet.command_metadata(page="demo", group="panel"):
pallet.fill_rect(20, 20, 240, 140, "#F8FAFC")
```

Reconnect Replay

RUNTIME AND DRAWING

When replay is enabled, the bridge stores a compact representation of current state. It coalesces high-frequency updates so reconnecting browsers do not receive stale frames.

- clear commands reset replay state for a page.
- chart_option with coalesce=True replaces earlier chart options for the same chart.
- UI updates and table updates are coalesced by target id.
- Page, chart, UI, terminal, and group deletes remove obsolete replay entries.

Coordinates And Sizing

RUNTIME AND DRAWING

Canvas coordinates are pixel-like drawing units relative to the browser surface. Most APIs accept x, y, width, and height. If width and height are omitted in Pallet.for_bridge(), the client uses the browser-reported drawing size.

- Keep chart rectangles large enough for labels and legends.
- Reserve padding for HTML overlay cards and table filters.
- Use fixed dimensions for dashboard regions that should not jump.
- For multi-page dashboards, reuse consistent coordinates across pages.

Drawing Primitives

RUNTIME AND DRAWING

The core Pallet API exposes immediate-mode primitives for lines, rectangles, circles, arcs, filled shapes, text, and paths. These commands are intentionally small and composable.

```
pallet.clear("#FFFFFF")
pallet.line(30, 30, 260, 70, "#2563EB", 4)
pallet.rect(30, 100, 180, 90, "#0F172A", 2)
pallet.fill_circle(310, 145, 45, "#F97316")
pallet.arc(440, 145, 55, 20, 300, "#059669", 8)
pallet.text(30, 230, "Immediate drawing", color="#0F172A", size=2)
```

Paths And Stroke Style

RUNTIME AND DRAWING

path() is the flexible primitive for polylines and shapes. It supports line caps, joins, and dash arrays. Use it when line(), rect(), and arc() are too limited.

```
pallet.path(
[(40, 60), (140, 20), (240, 90), (340, 35)],
color="#DC2626",
width=4,
line_cap="round",
line_join="round",
dash=[10, 6],
)
```

Text And Color

RUNTIME AND DRAWING

The browser accepts CSS color strings, including named colors, hex values, and rgba() values. Text uses a size scale rather than point sizes, so keep labels concise and test on the target display.

- Prefer high-contrast colors for operator dashboards.
- Use rgba() fills for annotation bands and translucent overlays.
- Use concise labels inside small cards and chart regions.
- Avoid relying on long strings in fixed-width controls.

Terminal Regions

UI AND TERMINALS

Terminal regions are scrollable text panes rendered in the browser. They are useful for logs, subprocess output, status streams, and mixed graph-plus-console demonstrations.

```
term = pallet.terminal_region(
"log",
x=24, y=420, width=760, height=180,
title="Worker log",
)
term.writeln("started", color="#16A34A")
term.write("waiting for samples...")
```

Pipe Terminal Example

UI AND TERMINALS

python/pallet_pipe_terminal.py runs a command and streams its output into a terminal region. Use it to make command-line tools visible beside charts or UI controls.

- Use --command to select the program to run.
- Use --region-id when multiple terminals share a page.
- Clear terminal output before a new run when old lines are not useful.
- Keep terminal panes tall enough for wrapped output.

Responsive Grids

UI AND TERMINALS

The UI overlay can define grids that hold cards. A grid arranges cards into columns with gap and padding values, then adapts when the browser gets narrower.

```
pallet.define_grid(
"dashboard",
columns=3,
min_column_width=280,
gap=16,
padding=16,
)
```

Cards

UI AND TERMINALS

Cards are named containers in a grid. They can hold controls, status widgets, tables, and ECharts charts. Use cards for repeated dashboard panels and grouped controls.

```
pallet.define_card("controls", grid="dashboard", title="Controls")
pallet.define_card(
"results",
grid="dashboard",
title="Results",
column_span=2,
)
```

Controls

UI AND TERMINALS

Supported controls include button, toggle, slider, select, text, and number. Controls emit browser events and can be updated from Python without rebuilding the card.

```
level = pallet.control(
"level",
card="controls",
kind="slider",
label="Level",
value=50,
minimum=0,
maximum=100,
step=1,
live=True,
)
level.set(75)
```

Control Events

UI AND TERMINALS

Callbacks run when your client polls events. Sliders with live=True emit input events while moving; other controls usually emit change events after a value is committed. Buttons emit click events.

```
def changed(event):
print(event["id"], event["event"], event.get("value"))

pallet.on_ui_event("*", changed)
while True:
event = pallet.poll_event(timeout=1.0)
```

Status Widgets

UI AND TERMINALS

Status widgets provide compact dashboard signals. Available kinds are badge, led, progress, kpi, alert, and spinner. Semantic states include info, success, warning, danger, and neutral.

```
load = pallet.status_widget(
"load",
card="results",
kind="progress",
label="CPU",
value=42,
units="%",
minimum=0,
maximum=100,
)
load.set(87, status="warning")
```

Data Tables

UI AND TERMINALS

Tables support keyed rows, live upserts, sorting, optional filtering, row selection, and row-click events. They are useful for recent samples, device lists, alerts, or audit trails.

```
table = pallet.table(
"sensors",
[{"key": "id", "label": "#", "align": "right"},
{"key": "name", "label": "Sensor"},
{"key": "value", "label": "Value", "align": "right"}],
card="results",
key_field="id",
filterable=True,
selectable=True,
)
table.upsert({"id": 1, "name": "A", "value": 43.7})
```

Dashboard Patterns

UI AND TERMINALS

A durable dashboard separates visual regions, control state, and update loops. Define grids and cards once, create widgets and charts once, then update values and series in place.

- Give every widget, table, terminal, and chart a stable id.
- Use pages for major views and cards for local grouping.
- Use coalesced updates for streams faster than a few frames per second.
- Keep control callbacks small; push heavy work into the main loop.

Graph Library Overview

CANVAS GRAPHS

pallet_graph_lib.py builds charts by issuing canvas commands through Pallet. It is well suited for deterministic, script-driven visuals that should share the same immediate drawing model as other canvas primitives.

- Graph handles numeric, datetime, log, and dual-y axes.
- Series include line, scatter, bar, histogram, area, confidence band, and heatmap.
- Other classes include ArcGauge, BarGauge, CircularMeter, and RadarChart.
- Graph updates can replace points or series without rebuilding every object.

Graph Quick Start

CANVAS GRAPHS

Create a Graph, add one or more series, then call draw(). The graph computes ranges, ticks, margins, labels, grid lines, legend placement, and data mapping.

```
import math
from pallet import Pallet
from pallet_graph_lib import Graph

xs = [index * 0.2 for index in range(40)]
with Pallet.for_bridge("127.0.0.1") as pallet:
graph = Graph(pallet, title="Sine", x_label="x", y_label="y")
graph.add_series(xs, [math.sin(x) for x in xs], label="sin")
graph.draw()
```

Axes And Ticks

CANVAS GRAPHS

Graph chooses rounded bounds and readable intervals using 1, 2, 2.5, and 5 style tick steps. Tick density adapts to available pixel space to reduce label collisions.

- Set x_min, x_max, y_min, and y_max for explicit ranges.
- Use y2_label and y_axis=2 for dual-y charts.
- Use grid_x and grid_y in GraphStyle to influence target interval counts.
- Ensure graph rectangles leave enough space for title, labels, and legend.

Datetime And Log Axes

CANVAS GRAPHS

Datetime x values are accepted as date or datetime objects. Naive datetimes are interpreted as UTC. Logarithmic x and y axes can be enabled independently, but log values must be positive.

```
from datetime import datetime, timedelta, timezone

start = datetime(2026, 6, 19, 8, tzinfo=timezone.utc)
times = [start + timedelta(minutes=30 * i) for i in range(12)]

graph = Graph(pallet, title="History", x_label="UTC")
graph.add_series(times, values, label="sensor")
graph.draw()
```

Lines And Scatter

CANVAS GRAPHS

add_series() supports line charts, scatter charts, or both. Use draw_line and draw_markers to select the visual mode. Use marker_radius, color, label, and line_width for presentation.

```
graph.add_series(
xs,
ys,
label="samples",
color="#2563EB",
draw_line=True,
draw_markers=True,
marker_radius=4,
)
```

Line Styles And Splines

CANVAS GRAPHS

Named line styles are solid, dashed, dotted, and dashdot. You can also supply a custom dash_pattern. Spline fitting draws a smooth interpolating curve while markers remain at original samples.

```
graph.add_series(
xs,
ys,
label="estimate",
draw_markers=False,
line_style="dashdot",
line_cap="round",
spline=True,
spline_resolution=16,
)
```

Gaps And Confidence Bands

CANVAS GRAPHS

Use None or NaN for intentional missing values. Lines, splines, areas, and confidence bands do not connect across missing samples. Confidence bands take lower and upper arrays for each x coordinate.

```
graph.add_confidence_band(
xs,
lower,
upper,
label="95% interval",
fill="rgba(56, 189, 248, 0.20)",
)
graph.add_series(xs, mean, label="estimate", draw_markers=False)
```

Bars And Histograms

CANVAS GRAPHS

Bar series can be grouped or stacked. Series with the same stack name accumulate; different stacks form side-by-side groups. Histograms can use an integer bin count or explicit bin edges.

```
graph.add_bar_series(quarters, retail_a, label="A retail", stack="A")
graph.add_bar_series(quarters, online_a, label="A online", stack="A")
graph.add_histogram(samples, bins=20, value_range=(-4, 4))
graph.draw()
```

Areas And Heatmaps

CANVAS GRAPHS

Area series fill below a line and can be stacked by giving them the same stack name. Heatmaps accept a matrix plus optional x and y edges and a color map.

```
graph.add_area_series(xs, received, label="RX", stack="traffic")
graph.add_area_series(xs, transmitted, label="TX", stack="traffic")

graph.add_heatmap(matrix, color_map=["#F8FAFC", "#67E8F9", "#2563EB"])
```

Radar And Polar Charts

CANVAS GRAPHS

RadarChart is an alias of PolarChart. It expects at least three categories and one or more series. Use minimum, maximum, levels, and start_angle to shape the scale.

```
from pallet_graph_lib import RadarChart

chart = RadarChart(pallet, ["Speed", "Range", "Comfort"], maximum=10)
chart.add_series([8, 6, 7], label="Model A", fill="rgba(37,99,235,0.18)")
chart.draw()
```

Canvas Gauges

CANVAS GRAPHS

The canvas graph library includes ArcGauge, CircularMeter, and BarGauge. Use GaugeStyle for reusable colors, labels, units, thresholds, and ticks.

```
from pallet_graph_lib import ArcGauge, GaugeStyle

ArcGauge(
pallet,
value=76,
minimum=0,
maximum=100,
title="CPU",
units="%",
style=GaugeStyle(fill_color="#0EA5E9"),
).draw()
```

Annotations

CANVAS GRAPHS

Annotations are chainable and render with the graph. Use spans for regions, hline and vline for limits and events, and point labels for individual values.

```
graph.add_y_span(70, 100, fill="rgba(248,113,113,0.20)", text="warning")
graph.add_x_span(4, 6, fill="rgba(250,204,21,0.22)", text="event")
graph.add_hline(70, text="limit", color="#DC2626")
graph.add_point_label(8, 82, "peak")
```

Live Canvas Updates

CANVAS GRAPHS

Label series uniquely when updating by name. Update methods accept redraw_axes='auto', True, or False. Auto redraws axes only when the calculated range changes.

```
graph.set_point("sensor", 4, y=27.5)
graph.append_point("sensor", 20, 31.2, max_points=100)
graph.append_points("sensor", [21, 22], [30.8, 29.7])
graph.shift_series("sensor", dx=1)
graph.trim_series("sensor", 50)
```

ECharts Overview

APACHE ECHARTS

Apache ECharts support is implemented in echarts_graph_lib.py. It is the best choice when you want browser-native tooltips, legends, data zoom, animated setOption updates, and chart types that ECharts already implements well.

- Use high-level helpers for common chart families.
- Use builders for structured multi-axis line charts.
- Use raw chart() when you need the complete ECharts option model.
- Use ChartHandle methods for incremental updates.

ECharts Quick Start

APACHE ECHARTS

Create an EChartsPallet, start it, clear a page, define a chart, show the page, and keep the process alive while the browser displays it.

```
from echarts_graph_lib import EChartsPallet

pallet = EChartsPallet()
pallet.start()
pallet.clear(page="echarts")
pallet.line_chart(
id="temperature",
x=24, y=24, width=720, height=360,
x_data=["08:00", "09:00", "10:00"],
y_data=[72.1, 72.8, 73.4],
title="Temperature",
smooth=True,
page="echarts",
)
pallet.show_page("echarts")
```

Raw ECharts Options

APACHE ECHARTS

chart() sends a normal ECharts option dictionary to the browser. This is the escape hatch for chart types, plugins, axes, labels, tooltip formatters, or visual maps not wrapped by helper methods.

```
handle = pallet.chart(
id="raw-bars",
x=20, y=20, width=640, height=360,
option={
"tooltip": {"trigger": "axis"},
"xAxis": {"type": "category", "data": ["A", "B", "C"]},
"yAxis": {"type": "value"},
"series": [{"type": "bar", "data": [12, 19, 8]}],
},
)
```

ChartHandle Updates

APACHE ECHARTS

Every ECharts chart returns a ChartHandle. The handle remembers the chart id and page, so update calls can stay concise. Use coalesce=True for high-frequency set_option loops.

```
handle.set_option(
{"series": [{"name": "Signal", "data": values}]},
coalesce=True,
)
handle.set_data([{"value": 82, "name": "Load"}])
handle.append_data([[5, 17]])
handle.resize(width=800, height=420)
handle.remove()
```

High-level ECharts Helpers

APACHE ECHARTS

The base helpers cover line_chart, bar_chart, pie_chart, gauge, and update_gauge. They build practical defaults for axes, grid, legends, labels, tooltips, and series data.

```
pallet.bar_chart(
id="utilization",
x=20, y=20, width=420, height=300,
categories=["CPU", "RAM", "Disk"],
values=[65, 72, 54],
horizontal=True,
series_name="Percent",
)
```

Gallery-style Helpers

APACHE ECHARTS

Additional helpers mirror common ECharts gallery patterns: progress_gauge, multi_ring_gauge, stacked_line_chart, area_line_chart, stepped_line_chart, bar_race_chart, doughnut_chart, rose_pie_chart, bubble_scatter_chart, heatmap_chart, radar_chart, candlestick_chart, funnel_chart, and treemap_chart.

- Use extra_option when helper defaults need small overrides.
- Use raw chart() when the data model differs from the helper.
- Keep ids stable so reconnect replay and later updates target the same chart.
- Prefer helper methods for prototypes, then drop to raw options only where needed.

Multi-axis Builder

APACHE ECHARTS

multi_axis_line_chart() hides ECharts axis indexes. Add named axes, attach named line series to those axes, then render. It is ideal for dashboards with top and bottom x axes or left and right y axes.

```
chart = pallet.multi_axis_line_chart(
id="plant", x=20, y=20, width=760, height=420,
title="Plant telemetry", data_zoom=True,
)
chart.add_x_axis("Time", data=["0s", "1s"], position="bottom")
chart.add_y_axis("Temperature", units="deg F", position="left")
chart.add_line("Temperature", [72.0, 72.5], x_axis="Time", y_axis="Temperature")
handle = chart.render()
```

Live Time Charts

APACHE ECHARTS

live_time_chart() creates a rolling time-series chart. It accepts a mapping from series name to ECharts series kind, appends timestamped values, and coalesces browser updates automatically.

```
from datetime import datetime, timezone

live = pallet.live_time_chart(
id="live",
x=20, y=20, width=720, height=360,
series={"Actual": "line", "Target": "bar"},
max_points=60,
)
live.append(datetime.now(timezone.utc), {"Actual": 42.5, "Target": 40.0})
```

ECharts In Cards

APACHE ECHARTS

ECharts charts can be positioned directly or hosted inside UI cards. When using cards, the card owns the browser layout while the chart still uses the id, page, and update APIs from EChartsPallet.

```
pallet.define_grid("dashboard", columns=2, gap=16, padding=16, page="dash")
pallet.define_card("left", grid="dashboard", title="Production", page="dash")
pallet.line_chart(
id="production",
x=0, y=0, width=480, height=300,
x_data=["A", "B", "C"],
y_data=[10, 14, 12],
card="left",
page="dash",
)
```

Example Programs

OPERATIONS

The repository includes runnable examples that exercise the feature set. Use them as smoke tests and as copyable starting points.

- python/pallet_graph_examples.py lists and runs canvas graph demos.
- python/pallet_ui_examples.py demonstrates grids, cards, controls, status widgets, and tables.
- python/echarts_examples.py shows gallery-style ECharts helpers.
- python/echarts_multi_axis_api.py shows the multi-axis builder.
- python/echarts_live_line_coalesce.py shows fast coalesced updates.
- python/echarts_live_grouped_bar.py is a larger live dashboard example.

Bridge CLI

OPERATIONS

The bridge accepts command-line options for host, TCP port, WebSocket port, and reconnect replay behavior. Use explicit ports when running multiple bridge instances.

```
python python/bridge.py --tcp-host 127.0.0.1 --tcp-port 9000 --ws-port 8080
python python/bridge.py --no-replay-on-connect
python python/bridge.py --replay-on-connect
```

Command Protocol

OPERATIONS

At the lowest level, clients send JSON command objects. The Pallet class wraps this protocol, but command() and commands() remain available for custom integrations.

- Drawing commands include clear, line, rect, fill_rect, circle, arc, text, and path.
- UI commands include ui_grid, ui_card, ui_control, ui_status, and ui_table.
- Terminal commands define, write to, and clear terminal regions.
- Chart commands define charts, set options, update data, append data, resize, and remove.

Troubleshooting Connections

OPERATIONS

Most connection issues come from the bridge not running, the browser not connected to the WebSocket URL, port mismatches, or a firewall blocking local sockets.

- Confirm python/bridge.py is running.
- Confirm pallet.html is connected to ws://localhost:8080.
- Confirm Python clients are using the bridge TCP port, usually 9000.
- Call pallet.status() or check EChartsPallet.client_count when debugging.
- If the browser reloads, rely on replay or rerun the client script.

Performance Guidelines

OPERATIONS

Use the smallest update that expresses the change. Avoid full redraws for high-frequency streams when a set_data, set_option, or table upsert is enough.

- Use coalesce=True for rapid ECharts set_option updates.
- Use Graph append and set methods instead of rebuilding large graphs where possible.
- Batch related canvas commands.
- Keep tables bounded with max_rows for long-running dashboards.
- Use pages to isolate heavy dashboards that do not need to be visible.

Visual Design Guidelines

OPERATIONS

Web Pallet dashboards work best when they are dense but calm. Use strong hierarchy, stable coordinates, concise labels, and consistent colors. Reserve bright colors for status changes and alerts.

- Give controls and charts predictable positions.
- Avoid tiny chart rectangles with long axis labels.
- Use semantic status colors consistently.
- Keep terminal panes and tables readable at the target display size.
- Use ECharts dataZoom when the user needs to inspect a long series.

Network And Safety

OPERATIONS

Web Pallet is typically a local development and operator-display tool. Treat the bridge as a command surface: anyone who can connect to it can draw, define UI, and stream terminal text.

- Bind to localhost unless remote clients are required.
- Be careful exposing bridge ports on shared networks.
- Do not stream secrets into terminal regions or tables.
- Review raw script_load usage before loading external scripts.
- Prefer explicit host and port settings in shared demos.

Extending Web Pallet

EXTENSION AND REFERENCE

Extension work usually starts in one of three places: a new Python helper that emits existing commands, a new browser command handler in pallet.html, or a new bridge coalescing rule for replay state.

- Start with Python helpers when the browser can already render the needed primitive.
- Add browser handlers when a new visual or DOM capability is required.
- Add bridge memory rules when reconnect replay should preserve only compact state.
- Keep command payloads JSON-serializable and stable.

Adding A Chart Helper

EXTENSION AND REFERENCE

A good helper hides repetitive option structure but leaves an escape hatch. Follow the ECharts helpers: accept clear Python data, build a normal option dictionary, merge extra_option, and return ChartHandle.

```
def my_chart(pallet, *, id, x, y, width, height, data, extra_option=None):
option = {
"tooltip": {"trigger": "item"},
"series": [{"type": "pie", "data": list(data)}],
}
option.update(extra_option or {})
return pallet.chart(id=id, x=x, y=y, width=width, height=height, option=option)
```

Testing Checklist

EXTENSION AND REFERENCE

Before sharing a dashboard or demo, run a browser smoke test and verify the reconnect path. Reload the browser after the script has drawn its state and confirm the bridge replays the expected current view.

- Run the relevant example script from the repository root.
- Open pallet.html and connect to ws://localhost:8080.
- Check visible layout at the target browser size.
- Interact with controls and verify event callbacks.
- Reload the browser to check replay behavior.
- Stop the client and bridge cleanly.

Pallet API Reference

EXTENSION AND REFERENCE

The general Pallet API covers lifecycle, raw commands, pages, batching, UI, tables, terminals, and drawing primitives.

- Lifecycle: for_bridge, connect, close, status, command, commands.
- Metadata: command_metadata, capture_commands, coalesce_group, replace_group.
- Pages: set_page, show_page.
- Events: subscribe_events, on_ui_event, poll_event, run_event_loop.
- UI: define_grid, define_card, control, update_control, status_widget, table.
- Terminal: terminal_region, write_terminal, clear_terminal.
- Drawing: clear, fill_screen, line, hline, vline, rect, fill_rect, circle, fill_circle, arc, text, path.

Graph API Reference

EXTENSION AND REFERENCE

The canvas graph library centers on Graph, GraphStyle, GaugeStyle, PolarChart, RadarChart, ArcGauge, BarGauge, and CircularMeter.

- Series: add_series, add_bar_series, add_histogram, add_area_series, add_confidence_band, add_heatmap.
- Annotations: add_vline, add_hline, add_x_span, add_y_span, add_point_label.
- Updates: set_series, set_confidence_band, set_point, append_point, append_points, shift_series, trim_series.
- Styling: GraphStyle controls backgrounds, axes, grids, labels, title colors, tick targets, and palette behavior.
- Gauges: GaugeStyle plus thresholds, labels, units, bounds, and orientation.

ECharts API Reference

EXTENSION AND REFERENCE

The ECharts layer centers on EChartsPallet, ChartHandle, LiveTimeChart, MultiAxisLineChart, Axis, and LineSeries.

- Generic: chart, set_option, set_data, append_data, resize_chart, remove_chart.
- Base helpers: line_chart, bar_chart, pie_chart, gauge, update_gauge.
- Gallery helpers: progress_gauge, multi_ring_gauge, stacked_line_chart, area_line_chart, stepped_line_chart, bar_race_chart, doughnut_chart, rose_pie_chart, bubble_scatter_chart, heatmap_chart, radar_chart, candlestick_chart, funnel_chart, treemap_chart.
- Builders: multi_axis_line_chart, line_chart_2x2y, add_x_axis, add_y_axis, add_line, render, update_line, update_x_axis.
- Live: live_time_chart and LiveTimeChart.append.

Glossary

EXTENSION AND REFERENCE

A few terms appear throughout the project. Keeping them straight makes it easier to reason about where a feature belongs.

- Browser surface: pallet.html and its rendering layers.
- Bridge: the process connecting TCP clients to browser WebSocket clients.
- Page: a named visual workspace inside the browser.
- Group: a named collection of commands that can be replaced together.
- Card: an HTML overlay container inside a grid.
- ChartHandle: an ECharts object used for later updates.
- Replay: compact bridge state sent to a reconnecting browser.

Release Checklist

EXTENSION AND REFERENCE

Use this checklist when preparing a reusable Web Pallet demo, dashboard, or helper module.

- Document how to start the bridge and which page to show.
- Use stable ids for charts, controls, tables, terminals, and cards.
- Bound live data structures such as chart windows and table rows.
- Use coalesced updates for high-frequency chart changes.
- Test browser reload and reconnect replay.
- Keep secrets out of terminal and table output.
- Include a small example script that runs from the repository root.